

```

import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.datasets import mnist, fashion_mnist
from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
tf.keras.utils.disable_interactive_logging()
print("--- Part 1: Building a Simple Feedforward Neural Network ---")
(x_train_fnn, y_train_fnn), (x_test_fnn, y_test_fnn) = fashion_mnist.load_data()
print("Original FNN training data shape:", x_train_fnn.shape)
print("Original FNN test data shape:", x_test_fnn.shape)
x_train_fnn_flat = x_train_fnn.reshape(-1, 28 * 28)
x_test_fnn_flat = x_test_fnn.reshape(-1, 28 * 28)
x_train_fnn_norm = x_train_fnn_flat.astype("float32") / 255.0
x_test_fnn_norm = x_test_fnn_flat.astype("float32") / 255.0
print("Flattened & Normalized FNN training data shape:", x_train_fnn_norm.shape)
print("Flattened & Normalized FNN test data shape:", x_test_fnn_norm.shape)
model_fnn = keras.Sequential([
    layers.Input(shape=(784,)),
    layers.Dense(128, activation="relu"),
    layers.Dropout(0.2),
    layers.Dense(64, activation="relu"),
    layers.Dense(10, activation="softmax")
])
model_fnn.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)
print("\n--- FNN Model Summary ---")
model_fnn.summary()

```

```

)
print("\n--- FNN Model Summary ---")
model_fnn.summary()
print("\n--- Training FNN Model ---")
history_fnn = model_fnn.fit(
    x_train_fnn_norm,
    y_train_fnn,
    epochs=10,
    validation_split=0.1,
    verbose=1
)
print("\n--- Evaluating FNN Model ---")
loss_fnn, accuracy_fnn = model_fnn.evaluate(
    x_test_fnn_norm,
    y_test_fnn,
    verbose=0
)
print("FNN Test Loss:", round(loss_fnn, 4))
print("FNN Test Accuracy:", round(accuracy_fnn, 4))
y_pred_fnn = np.argmax(
    model_fnn.predict(x_test_fnn_norm, verbose=0),
    axis=1
)
print("\n--- FNN Classification Report ---")
print(classification_report(y_test_fnn, y_pred_fnn))
print("\n--- FNN Confusion Matrix ---")
cm_fnn = confusion_matrix(y_test_fnn, y_pred_fnn)
plt.figure(figsize=(10, 8))
sns.heatmap(
    cm_fnn,
    annot=True,
    fmt="d",
    cmap="Blues",
    cbar=False
)

```

```

)
plt.title("FNN Confusion Matrix")
plt.xlabel("Predicted Label")
plt.ylabel("True Label")
plt.show()

plt.figure(figsize=(8, 5))
plt.plot(
    history_fnn.history["accuracy"],
    label="Training Accuracy"
)
plt.plot(
    history_fnn.history["val_accuracy"],
    label="Validation Accuracy"
)
plt.title("FNN Model Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)
plt.show()
plt.figure(figsize=(8, 5))
plt.plot(
    history_fnn.history["loss"],
    label="Training Loss"
)
plt.plot(
    history_fnn.history["val_loss"],
    label="Validation Loss"
)
plt.title("FNN Model Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()
print("\n--- Part 2: Implementing a CNN ---")
(x_train_cnn, y_train_cnn), (x_test_cnn, y_test_cnn) = mnist.load_data()

```

```
)  
plt.title("CNN Confusion Matrix")  
plt.xlabel("Predicted Label")  
plt.ylabel("True Label")  
plt.show()  
plt.figure(figsize=(8, 5))  
plt.plot(  
    history_cnn.history["accuracy"],  
    label="Training Accuracy"  
)  
plt.plot(  
    history_cnn.history["val_accuracy"],  
    label="Validation Accuracy"  
)  
plt.title("CNN Model Accuracy")  
plt.xlabel("Epoch")  
plt.ylabel("Accuracy")  
plt.legend()  
plt.grid(True)  
plt.show()  
plt.figure(figsize=(8, 5))  
plt.plot(  
    history_cnn.history["loss"],  
    label="Training Loss"  
)  
plt.plot(  
    history_cnn.history["val_loss"],  
    label="Validation Loss"  
)  
plt.title("CNN Model Loss")  
plt.xlabel("Epoch")  
plt.ylabel("Loss")  
plt.legend()  
plt.grid(True)
```

--- Part 1: Building a Simple Feedforward Neural Network ---

Original FNN training data shape: (60000, 28, 28)
Original FNN test data shape: (10000, 28, 28)
Flattened & Normalized FNN training data shape: (60000, 784)
Flattened & Normalized FNN test data shape: (10000, 784)

--- FNN Model Summary ---

--- Training FNN Model ---

--- Evaluating FNN Model ---

FNN Test Loss: 0.3367
FNN Test Accuracy: 0.8829

--- FNN Classification Report ---

	precision	recall	f1-score	support
0	0.85	0.82	0.83	1000
1	0.99	0.97	0.98	1000
2	0.78	0.81	0.80	1000
3	0.84	0.91	0.87	1000
4	0.77	0.83	0.80	1000
5	0.96	0.97	0.96	1000
6	0.76	0.63	0.69	1000
7	0.95	0.95	0.95	1000
8	0.97	0.98	0.97	1000
9	0.96	0.96	0.96	1000
accuracy			0.88	10000
macro avg	0.88	0.88	0.88	10000
weighted avg	0.88	0.88	0.88	10000

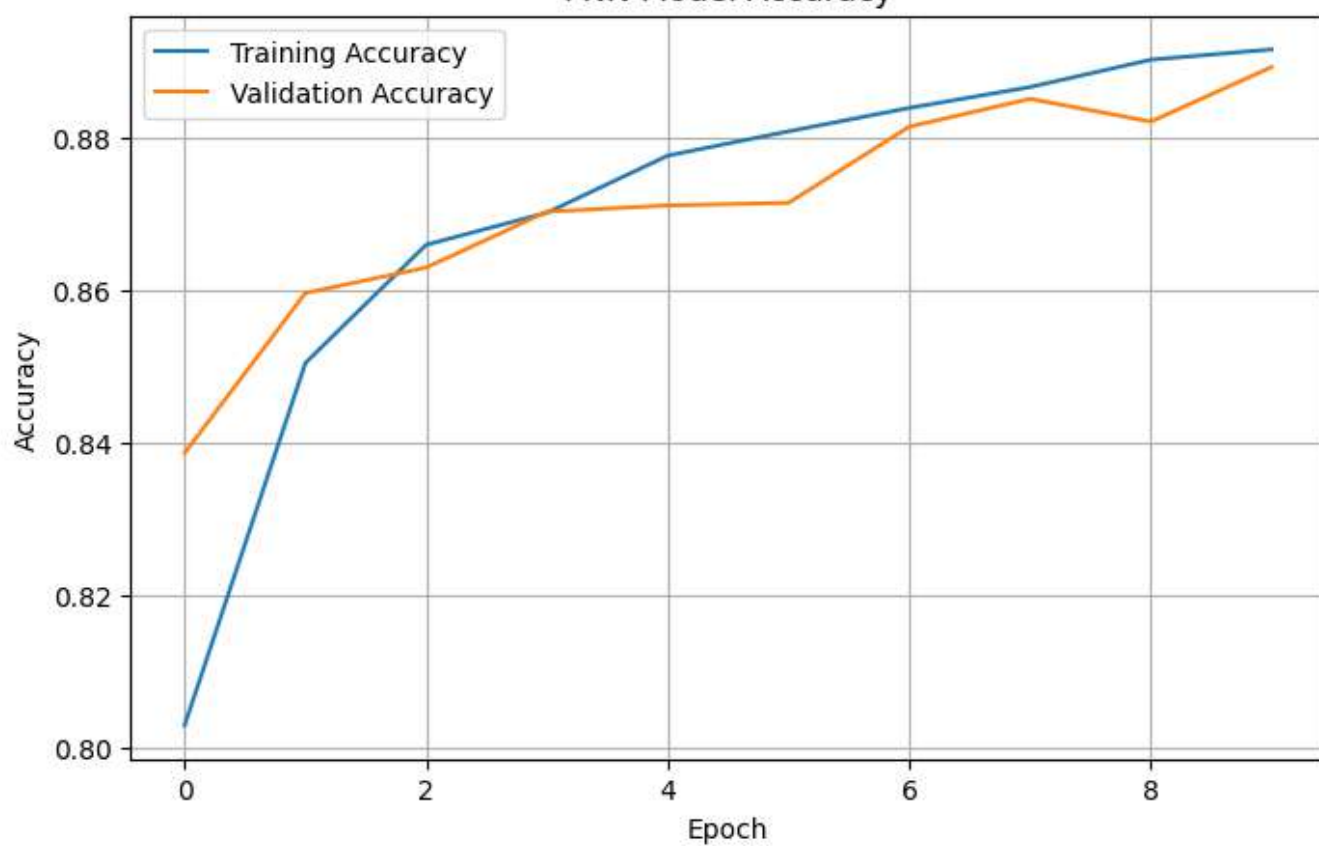
--- FNN Confusion Matrix ---

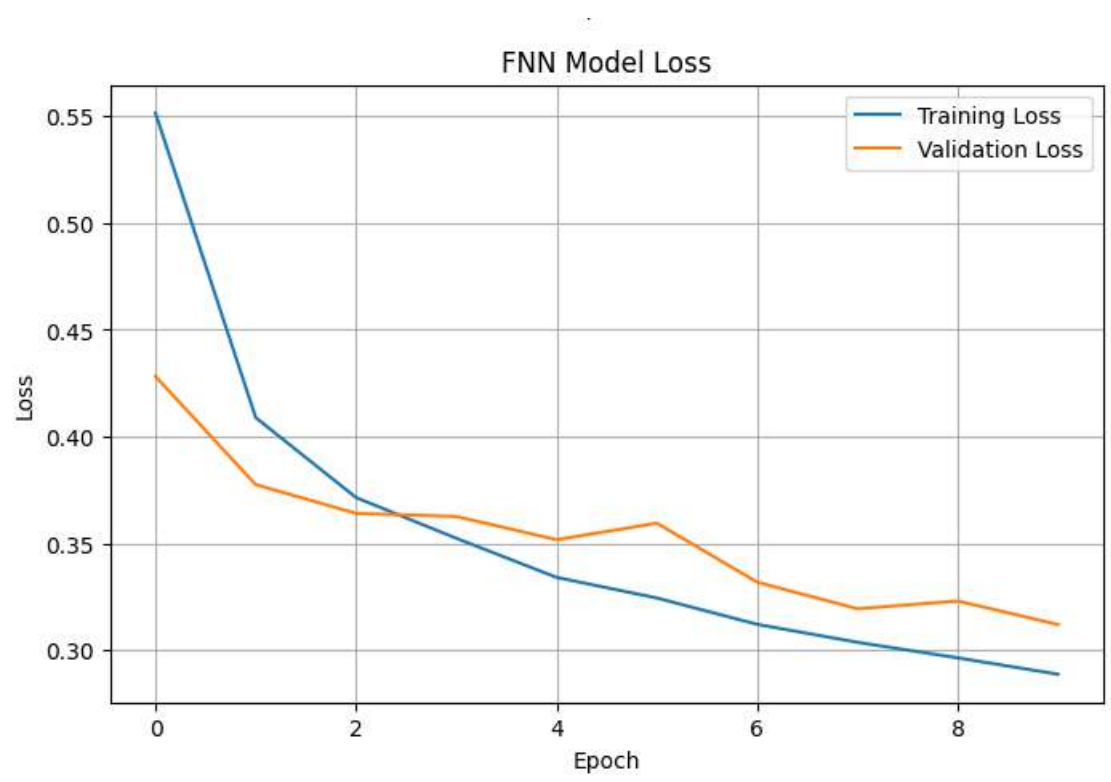
FNN Confusion Matrix

True Label	0	819	3	19	53	5	0	92	0	9	0
	1	1	971	0	21	4	0	1	0	2	0
	2	13	0	813	16	111	0	44	0	3	0
	3	11	9	8	908	45	1	15	0	3	0
	4	0	0	95	30	828	0	44	0	3	0
	5	0	0	0	0	0	972	0	17	1	10
	6	122	1	99	43	86	0	634	0	14	1
	7	0	0	0	0	0	27	0	948	0	25
	8	3	0	3	6	3	3	2	4	976	0
	9	0	0	0	0	0	12	1	27	0	960
		0	1	2	3	4	5	6	7	8	9
		Predicted Label									

FNN Model Accuracy

FNN Model Accuracy





```
--- Part 2: Implementing a CNN ---  
Original CNN training data shape: (60000, 28, 28)  
Original CNN test data shape: (10000, 28, 28)  
Reshaped & Normalized CNN training data shape: (60000, 28, 28, 1)  
Reshaped & Normalized CNN test data shape: (10000, 28, 28, 1)
```

```
--- CNN Model Summary ---
```

```
--- Training CNN Model ---
```